

A Practical Introduction to Retrieval-Augmented Generation (RAG)

How AI searches documents before answering.

Jernej Hribar

Institut Jožef Stefan, Slovenia,

Email: Jernej.hribar@ijs.si

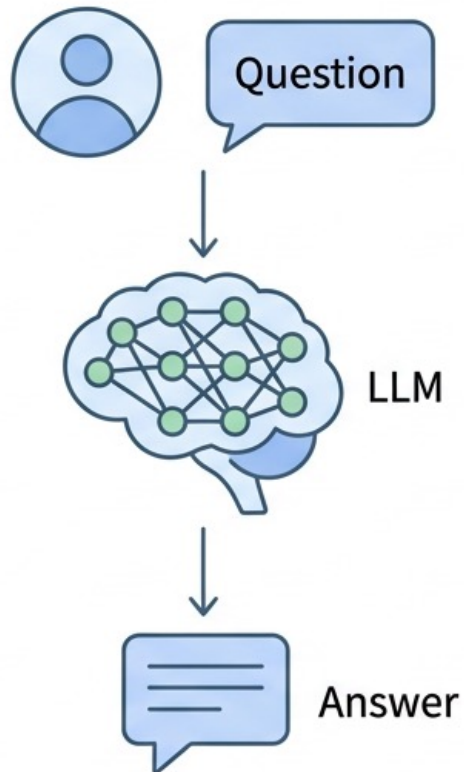
April 2026,

Munster Technological University, Ireland



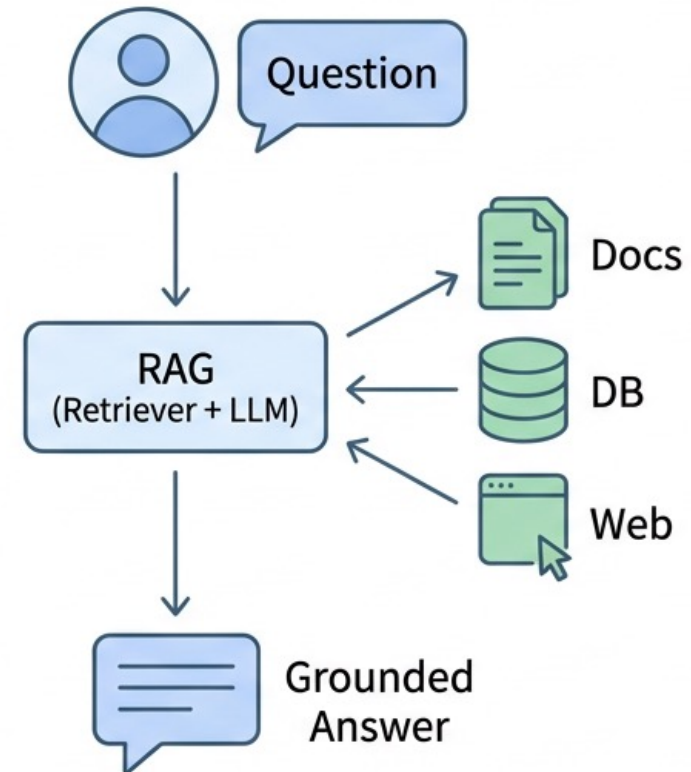
Standard LLMs vs RAG

Standard LLM



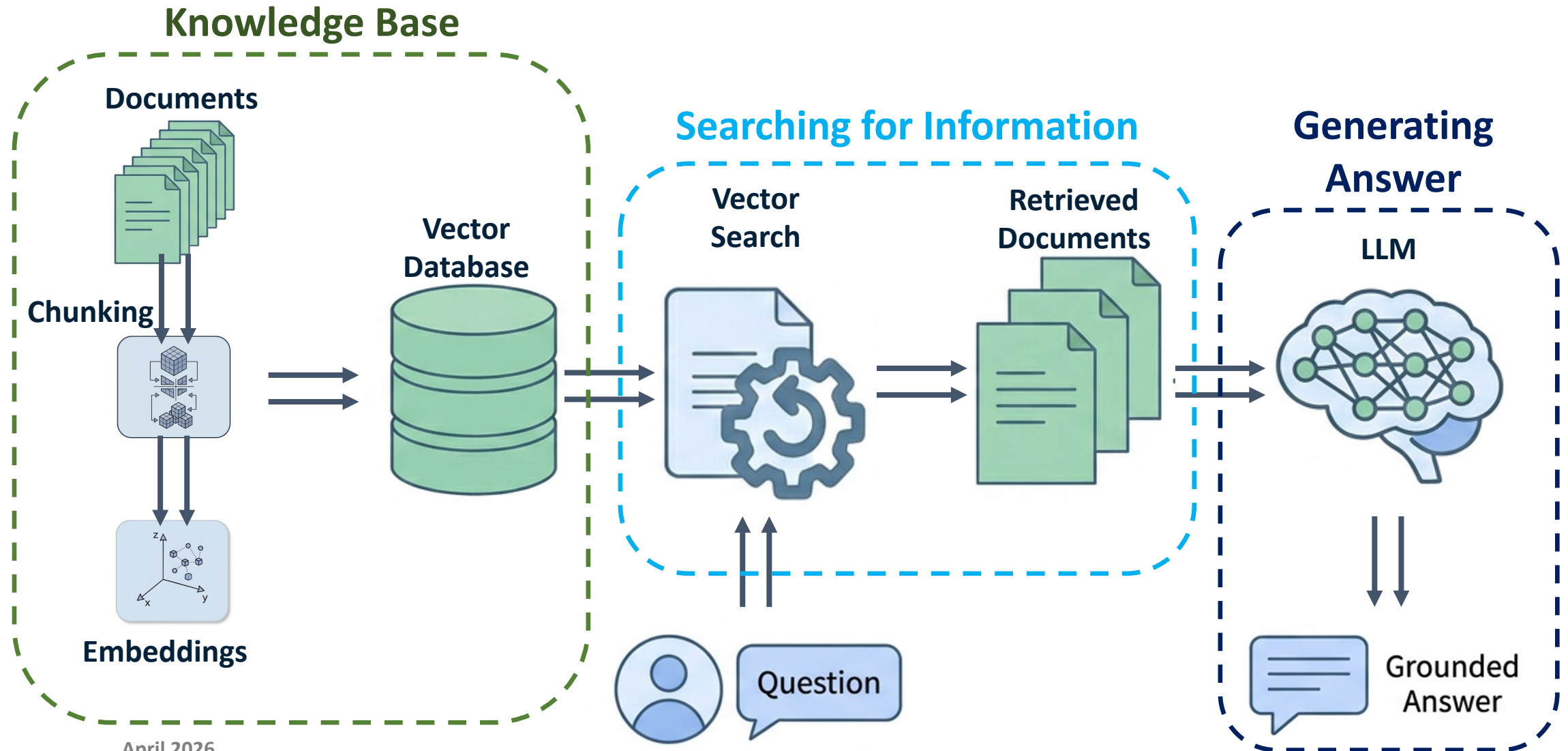
Answers from training data only

RAG-based System



Answers using your data + model

What is RAG?



When to use RAG

- Need **fresh** or frequently **changing knowledge** without retraining the model.
- Want **answers grounded** in specific sources, with evidence/provenance.
- Use private, domain-specific or regulated data that **must stay** in your control.
- Let **small** or local models **match big-model quality** by plugging into an external knowledge base.

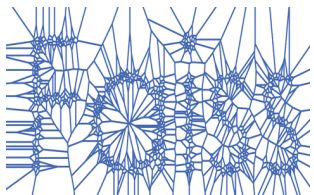
A stack of five light green rectangular papers with rounded corners and horizontal lines, fanned out from top-left to bottom-right.

Building Knowledge Base: Finding data

- Choose trusted sources and owners (systems of record, domain experts).
- Clean and deduplicate content; keep one canonical version per document.
- Apply permissions and classification (who can see what, sensitivity levels).
- Enrich documents with metadata for search (topic, product, customer, date, etc.).
- **RAG quality starts before retrieval.**



Building Knowledge Base: Selecting the database



FAISS for teaching and very small demos where everything can live in memory.



Chroma

Chroma DB for simple, local persistence and quick experiments on your laptop or a single server.

Postgres + PG Vector

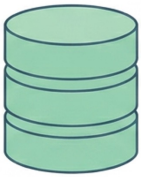


PG Vector if your data already lives in PostgreSQL and you want to keep everything in one database.



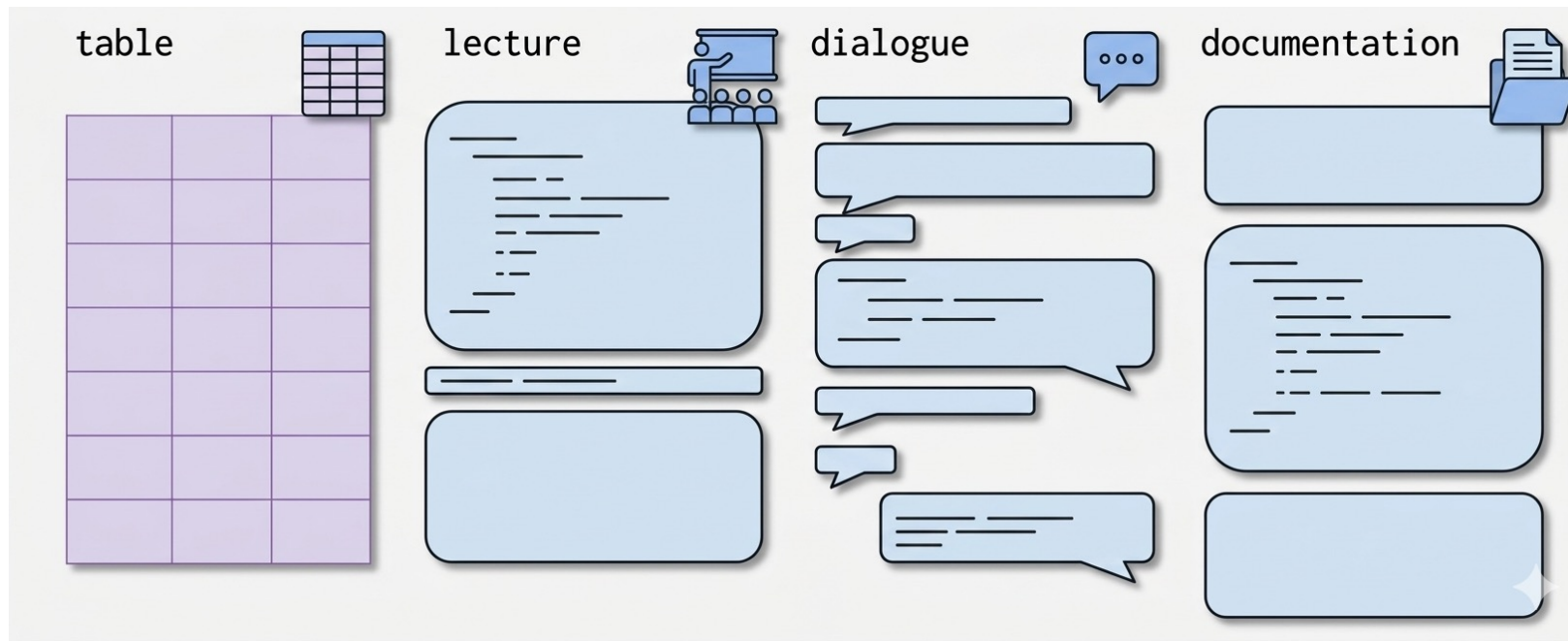
Pinecone

Pinecone / Qdrant for production-style retrieval with managed or scalable vector databases.



Building Knowledge Base: Chunking

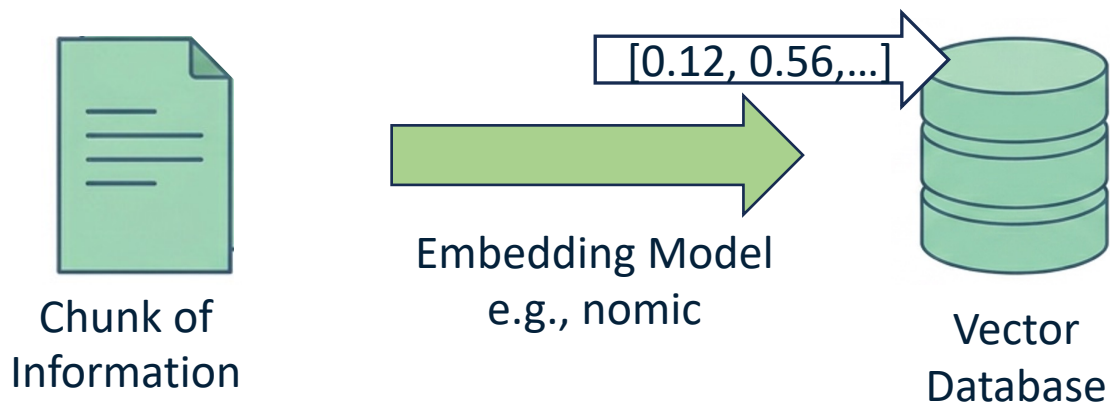
- **Break long documents into smaller retrievable units (“chunks”).**
- **Respect meaning and structure:** split on sections, headings, and paragraphs rather than arbitrary character counts.
- **Tune chunk size and overlap:** start with chunks around 200–500 tokens and ~10–20% overlap, then adjust based on retrieval quality.





Building Knowledge Base: Embeddings

- Embeddings **turn text** into numeric vectors that you store in a vector database.
- You **search** with cosine (or dot-product) similarity, or use these vectors inside RAG pipelines.
- **Vector length depends on the model** (often 384–1024 dimensions, sometimes higher).
- **Always use the same embedding model for indexing and querying.**

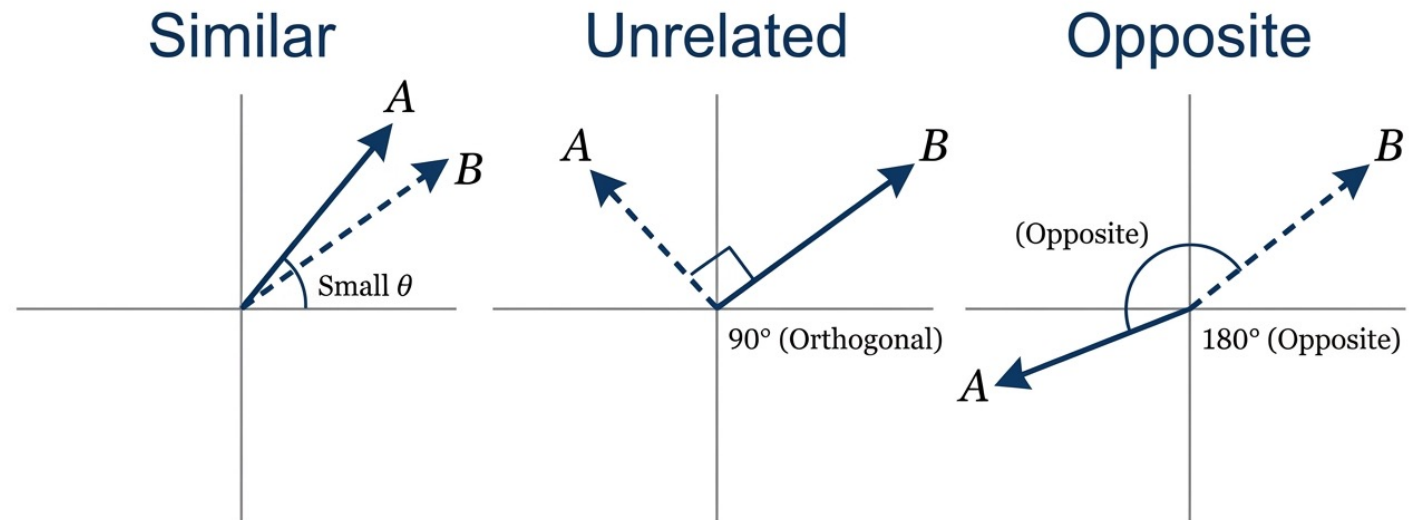




Searching for the Information: Vector search

- Use **top-k nearest-neighbour** search to return the k most similar chunks for a query vector. (k = 3–10 in many RAG setups.)

$$\cos(\theta) = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



- Combine **vector similarity with metadata** filters (tenant, document type, date, permissions) to keep results relevant and safe.



Answer Generation

- Combine the user question, retrieved chunks, and instructions into a single prompt and send it to the LLM.
- The LLM returns a grounded answer (and optionally citations) based on that context.
- Prompt Example:

You are a helpful assistant that answers using the context below.

Context:

{{retrieved_chunks}}

Question:

{{user_question}}

If the answer is not in the context, say you don't know.



LLM Model



Grounded Answer



Demo 1: Simple RAG with Ollama

- **Ollama** is an open-source tool that lets you run large language models (LLMs) locally on your own computer.
- Questions to ask:
 - **Easy semantic match:** What is RLHF?
 - **Rewrite needed:** What is PPO in RLHF?
 - **Exact technical term:** What is Proximal Policy Optimization?



Real-world RAG use cases

- **Internal knowledge assistant** – search across wikis, Confluence, and docs for employees, e.g., Bell (telecom) internal policy chatbot[1].
- **Customer support bot** – answer support questions from FAQs, tickets, and product manuals, e.g., Klarna handles 2/3 of customer support [2].
- **Compliance / policy assistant** – query regulations and internal policies with citations, e.g., pharma regulatory compliance [3].
- **Research assistant** – summarize and compare papers, reports, and studies, e.g., Gosearch [4].

[1] <https://www.evidentlyai.com/blog/rag-examples>

[2] <https://www.zenml.io/llmops-database/ai-assistant-for-global-customer-service-automation>

[3] Kim, Jaewoong, Minseok Hur, and Moohong Min. "From rag to qa-rag: Integrating generative ai for pharmaceutical regulatory compliance process." *In Proc. 40th ACM/SIGAPP 2025*.

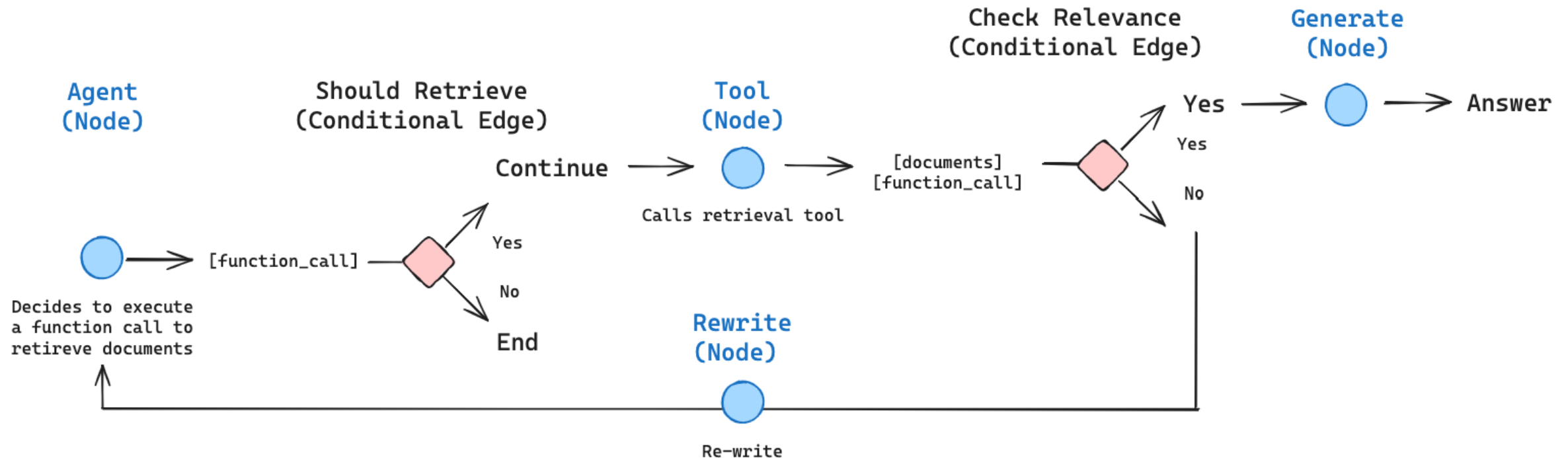
[4] <https://www.gosearch.ai/faqs/how-is-retrieval-augmented-generation-used-in-enterprise-search/>

Why naïve RAG is not enough

- **Chunking issues** – a single, one-shot retrieval over fixed chunks often misses cross-chunk context or multi-hop relationships, so answers are incomplete or brittle.
- **Synonym / query-understanding mismatch** – plain vector search struggles when the user asks in different words, or when the right answer requires reformulating the query or running multiple retrieval passes.
- **Single-shot, static pipeline** – naïve RAG “retrieve once, generate once”; it cannot plan, refine queries, or retrieve again when the first context is weak, so it fails on complex, multi-step questions.

Agentic RAG

We need something that can plan, adapt, and retrieve iteratively:
Agentic RAG.



Source: <https://docs.langchain.com/oss/python/langgraph/agentic-rag>



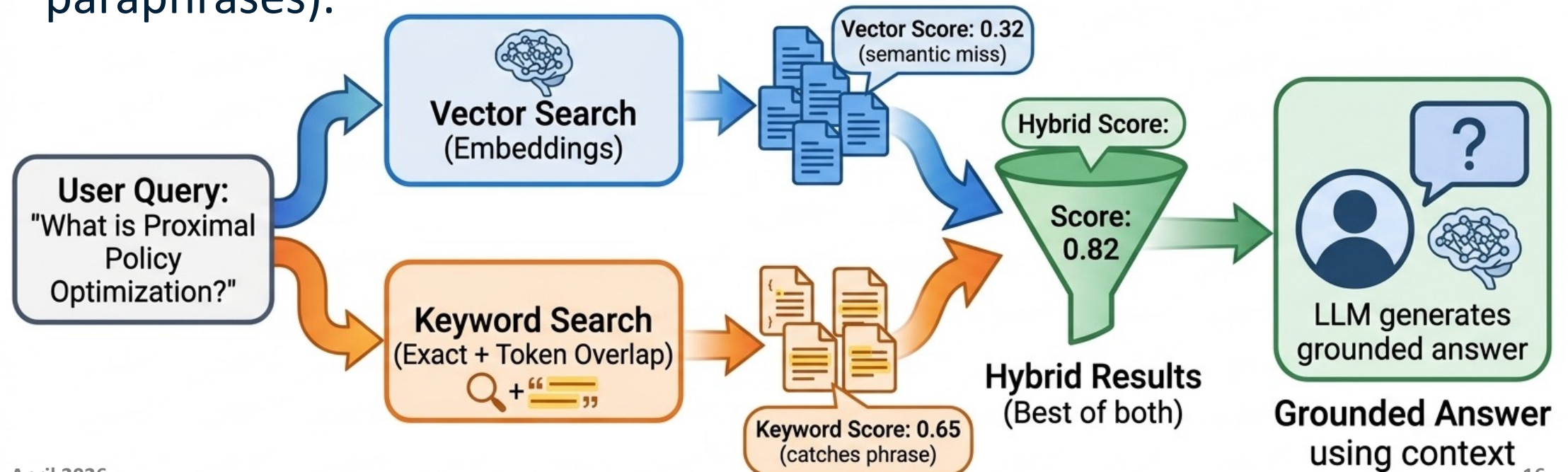
Demo 2: Agentic RAG with LangGraph

- *Self-correcting retrieval instead of one-shot retrieval*
- **Retrieve** relevant entries from the dataset
- **Grade** whether the retrieved context is strong enough
- **Rewrite** the question if the retrieval is weak
- **Retrieve again** using the improved query
- **Answer** only after the context passes the relevance check



Hybrid retrieval for smarter agents

- Hybrid retrieval combines semantic (vector) search and keyword (BM25) search to get the best of both worlds.
- Agentic RAG can pick hybrid retrieval when a query needs both exact terms (IDs, policy names) and semantic understanding (synonyms, paraphrases).





Demo 3: Hybrid Agentic Search

Semantic similarity + exact phrase matching

- **Hybrid retrieval:** Semantic search + keyword boost for exact terms
- **Failure case:** "What is Proximal Policy Optimization?"
- **Expected result:** Hybrid ranks the right answer #1, semantic-only misses it
- **Why hybrid matters:**
Long technical phrases like "Proximal Policy Optimization" are hard for pure embeddings to match exactly



When RAG should say “I don’t know”

- **No relevant evidence retrieved** – if the system cannot find trustworthy supporting chunks, it should abstain instead of guessing.
- **Conflicting evidence** – if retrieved documents disagree and the system cannot resolve the conflict, it should surface uncertainty.
- **Out-of-scope questions** – if the answer is outside the knowledge base or outside allowed capabilities, the system should not invent one.
- **Low confidence after multiple retrieval attempts** – even Agentic RAG should stop and say “I don’t know” when retries, reformulations, or hybrid search still do not produce grounded evidence.

Offline Evaluation

- **Build a test set of questions, expected answers, and relevant documents.** Offline evaluation starts with a dataset, not random manual testing.
- **Evaluate retrieval and generation separately.** Measure whether the retriever finds the right context first, then whether the LLM produces a grounded answer from that context.
- **Use tools such as Langfuse and RAGAS.** Langfuse helps manage datasets, traces, and experiments, while RAGAS provides reference-free metrics for RAG quality.
- **Compare experiments systematically.** Test chunking, embeddings, retrievers, rerankers, and prompts side by side instead of changing everything at once.

Monitoring in Production

- **Trace every request end to end** – user query, retrieved documents, prompts, model calls, and final answer.
- **Monitor key operational metrics** – latency, errors, token usage, and cost over time.
- **Track quality signals in production** – retrieval failures, low-confidence answers, user feedback, and score dashboards.
- **Tools:** Langfuse, Prometheus/Grafana, custom dashboards, or similar observability stacks.

Common Tools for Building RAG Systems

- **Visual builders:** Langflow, Flowise, n8n.
- **Frameworks:** LangChain, LangGraph, LlamaIndex, Haystack.
- **Model runtime / serving:** Ollama, OpenAI API, vLLM, Hugging Face Inference.
- **Vector stores:** FAISS, Chroma, pgvector, Qdrant, Pinecone, Weaviate, Milvus.
- **Evaluation / observability:** Langfuse, RAGAS, DeepEval.
- **Automation / ingestion:** Unstructured, Airbyte, n8n, custom ETL pipelines.

Future challenges and where to learn more

Challenges:

- Reliable evaluation
- Hallucination detection
- Latency vs. quality trade-offs
- Agent coordination and governance

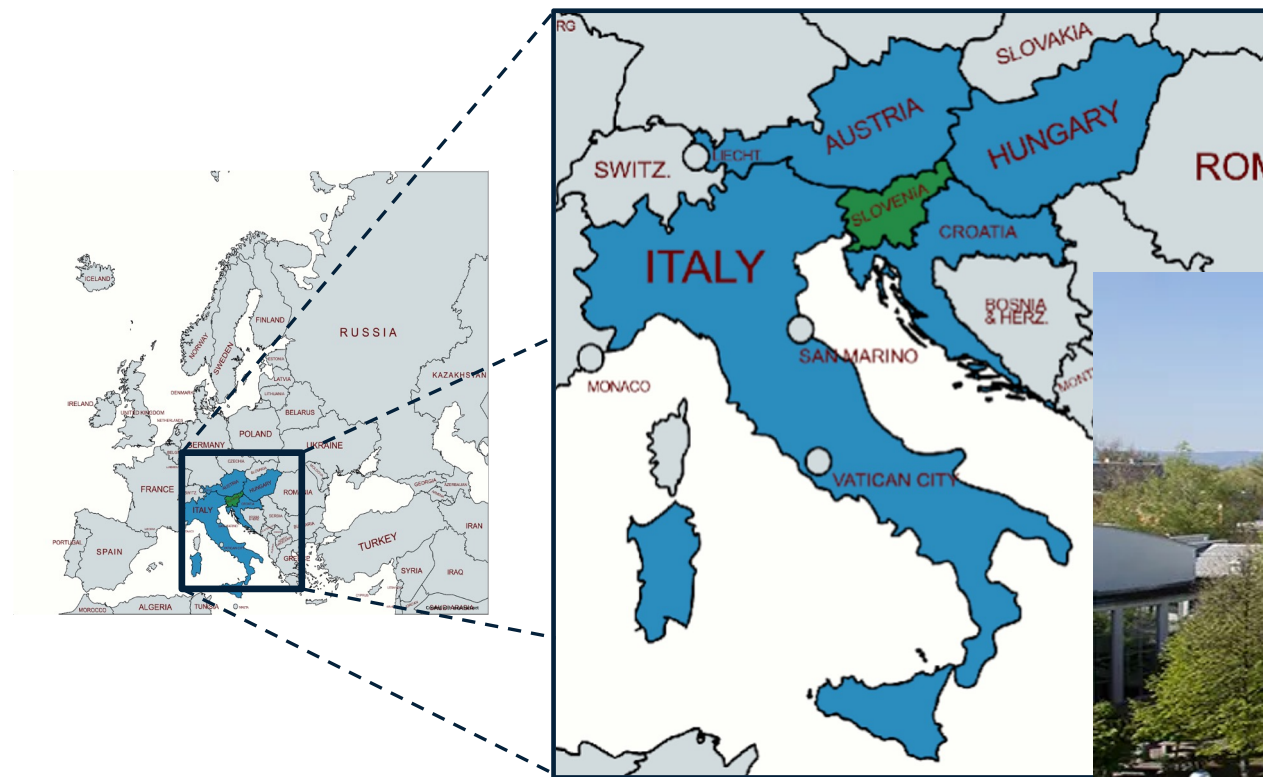
Reading list:

- Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG
- GaRAGe: A Benchmark with Grounding Annotations for RAG Evaluation
- Towards Agentic RAG with Deep Reasoning: A Survey of RAG-Reasoning Systems in LLMs

Thank you!

Email: jernej.hribar@ijs.si

Jožef Stefan Institute



 Institut
"Jožef Stefan"
Ljubljana, Slovenija